

03 stack

Stack Pattern

1. What is it, and when is it used?

A **stack** is a LIFO (Last-In-First-Out) structure supporting `push`, `pop`, and `peek` in $O(1)$. It's used whenever a problem has **nested structure, needs to "remember the most recent unresolved thing," or asks about the nearest greater/smaller element** in a sequence.

2. Why is it used?

Many problems require reversing the order of processing or matching things from the innermost outward (like parentheses). A stack naturally captures "the most recent thing I haven't dealt with yet," letting you resolve nested/sequential dependencies in $O(n)$ instead of $O(n^2)$ brute force.

3. Where is it used?

- Balanced parentheses / bracket validation
- Expression evaluation (infix→postfix, calculator problems)
- Next Greater Element / Next Smaller Element
- Largest Rectangle in Histogram, Trapping Rain Water
- Function call stack / recursion simulation (iterative DFS, backtracking)
- Undo operations, browser history
- Stock span problem, daily temperatures
- Decode strings like `"3 [a2 [c]] "`
- Min Stack ($O(1)$ `getMin`)
- Syntax parsing / compilers (bracket & operator matching)

4. How do you identify it's a Stack problem?

Signal phrases (2-minute test): - "valid parentheses" / "matching brackets" - "next greater element" / "next smaller element" / "previous greater/smaller" - "largest rectangle" in a histogram/skyline - "evaluate expression" (infix, postfix, prefix) - "remove adjacent duplicates" / "decode string" - "daily temperatures" (how many days until warmer) - Anything about "the most recent unmatched/unresolved element" - Nested structure: brackets within brackets, tags within tags (HTML/XML)

5. Can you identify it from constraints?

Constraint clue	Implication
"Find next/previous greater or smaller element for every index" and $O(n^2)$ brute force too slow	Monotonic stack $\rightarrow O(n)$
Nested symbols (<code>(){}[]</code>) need validation	Simple stack matching
Need $O(1)$ <code>getMin()</code> / <code>getMax()</code> alongside normal stack ops	Min/Max Stack (auxiliary stack)
Recursion depth could blow the call stack (very deep trees/graphs)	Convert recursion to explicit stack (iterative DFS)
Histogram/array where you want max area rectangle	Monotonic increasing stack pattern

6. Types / Variants

1. **Basic Stack** — push/pop/peek, used for matching & reversing order (bracket validation, string reversal).
2. **Monotonic Stack (increasing)** — keeps elements increasing bottom→top; pops when a smaller element

arrives. Used for “next smaller element,” histogram problems.

3. **Monotonic Stack (decreasing)** — keeps elements decreasing; pops when a larger element arrives. Used for “next greater element,” daily temperatures.
4. **Min/Max Stack** — auxiliary stack tracks running min/max alongside main stack for $O(1)$ queries.
5. **Two Stacks** — e.g., implement a queue using two stacks, or evaluate expressions (one for numbers, one for operators).
6. **Stack for Recursion Simulation** — explicit stack replacing call stack for iterative DFS/backtracking, avoiding stack overflow on deep recursion.

7. Rationale / Intuition

The core intuition of a **monotonic stack**: when scanning left to right, if the current element is smaller (or bigger, depending on variant) than the element at the top of the stack, that top element has just found its “answer” (its next smaller/greater element) — so you pop it and record the answer, then push the current element. Every element is pushed once and popped at most once, giving **amortized $O(n)$** instead of the $O(n^2)$ brute-force nested loop.

For matching problems (parentheses), the stack naturally mirrors the “last opened, first closed” nesting rule — you push on open symbols and pop-and-compare on close symbols.

8. Time & Space Complexity

Operation	Time	Space
Push/Pop/Peek	$O(1)$	$O(1)$ per op
Balanced parentheses check	$O(n)$	$O(n)$
Next Greater Element (monotonic stack)	$O(n)$ amortized	$O(n)$
Largest Rectangle in Histogram	$O(n)$	$O(n)$
Min Stack	$O(1)$ per op	$O(n)$
Iterative DFS with explicit stack	$O(V + E)$	$O(V)$

9. Comparison with Other Patterns

Pattern	Order processed	Best for	Complexity vs brute force
Stack	LIFO	Nesting, next greater/smaller, matching	$O(n)$ vs $O(n^2)$
Queue	FIFO	BFS, order-preserving processing	$O(n)$
Monotonic Deque	Both ends	Sliding window max/min	$O(n)$ vs $O(n \cdot k)$
Two Pointers	Sequential, both ends of array	Sorted array pair problems	$O(n)$
Recursion / DFS	LIFO implicitly (call stack)	Tree/graph traversal	Same as stack but implicit

Rule of thumb: if you find yourself wanting to “look back at the most recent unresolved item,” reach for a stack. If instead you need “the next few items in a sliding range,” reach for a monotonic deque.

10. Reusable Java Templates

10.1 Valid Parentheses

```
import java.util.*;

class ValidParentheses {
    public boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        Map<Character, Character> pairs = Map.of(')', '(', ')', '[', ']', '[');
    }
}
```

```

        for (char c : s.toCharArray()) {
            if (pairs.containsKey(c)) {
                stack.push(c);
            } else if (pairs.containsValue(c)) {
                if (stack.isEmpty() || stack.pop() != pairs.get(c)) return false;
            }
        }
        return stack.isEmpty();
    }
}

```

10.2 Monotonic Stack — Next Greater Element

```

import java.util.*;

class NextGreaterElement {
    public int[] nextGreater(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        Arrays.fill(result, -1);
        Deque<Integer> stack = new ArrayDeque<>(); // stores indices, decreasing values

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
                result[stack.pop()] = nums[i];
            }
            stack.push(i);
        }
        return result;
    }
}

```

10.3 Min Stack (O(1) getMin)

```

import java.util.*;

class MinStack {
    private final Deque<Long> stack = new ArrayDeque<>(); // stores (val - min) trick, or use pair stack
    private long min;

    public void push(int val) {
        if (stack.isEmpty()) { stack.push(0L); min = val; }
        else {
            stack.push((long) val - min);
            if (val < min) min = val;
        }
    }

    public void pop() {
        long top = stack.pop();
        if (top < 0) min = min - top; // restore previous min
    }

    public int top() {
        long top = stack.peek();
        return (int) (top < 0 ? min : top + min);
    }

    public int getMin() { return (int) min; }
}

```

10.4 Largest Rectangle in Histogram

```

import java.util.*;

class LargestRectangle {
    public int largestRectangleArea(int[] heights) {
        Deque<Integer> stack = new ArrayDeque<>(); // increasing indices by height
        int maxArea = 0, n = heights.length;

        for (int i = 0; i <= n; i++) {
            int h = (i == n) ? 0 : heights[i];
            while (!stack.isEmpty() && heights[stack.peek()] >= h) {
                int height = heights[stack.pop()];
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                maxArea = Math.max(maxArea, height * width);
            }
            stack.push(i);
        }
        return maxArea;
    }
}

```

11. Quick Recognition Checklist (2-minute test)

- ☐ Brackets/parentheses/tags need matching or validating?
- ☐ “Next/previous greater or smaller element” phrasing?
- ☐ Need to undo the last operation or peek at “most recent unresolved” item?
- ☐ Nested/recursive structure that would otherwise need $O(n^2)$ look-back?

If 2+ checked → Stack (plain or monotonic).